Predictive Analytics

Predictive analytics

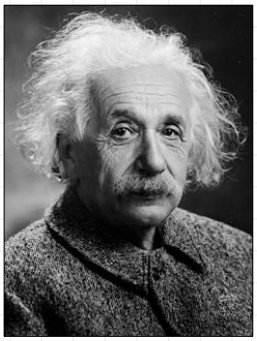
Predictive analytics is an area of statistics that deals with extracting information from data and using it to predict trends and behavior patterns.

The core of predictive analytics relies on capturing relationships between explanatory variables and the predicted variables from past occurrences and exploiting them to predict the unknown outcome.

https://en.wikipedia.org/wiki/Predictive_analytics

Prediction is very difficult, especially about the future

Everything Should Be Made as Simple as Possible, But Not Simpler



Predictive analytics

Predictive

- Primary objective is the prediction of future events on the basis of data (time series or other) available *now*.

Analytics

- Forecasts are used to take decisions *now.*

Examples (management)

- Forecast of goods or services demand
- Forecast of workforce availability or need
- Forecast of material requirements and of warehouse availabilities.
- Forecast of revenues, profits and losses in order to define future investments

Forecast horizons

short term (*days, weeks*)

- Resource requirements
- Workforce shifts
- Retail sales

medium term (*weeks, months*)

- Sales by product category
- Workforce needs
- Resource requirements

long term (*months, years*)

- Growth trends
- Storage needs
- Sales patterns, market trends

Forecasts, attributes

They are usually wrong.

Forecasts are usually not just numbers:

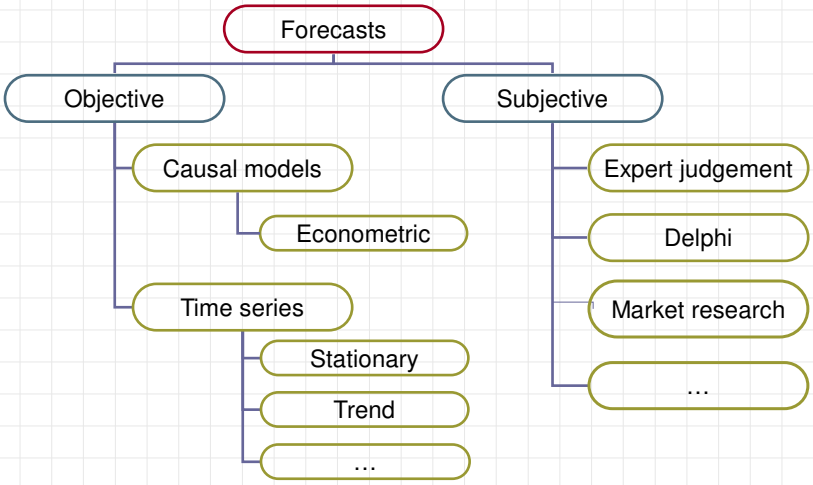
- They include *mean and dispersion measures*
- They include a *confidence interval*

Usually, aggregate data forecasts are more reliable than detail data ones.

Usually reliability decreases with the distance in the future.

Prediction models

Several types of models for data forecast



Subjective models

Very varied, they include :

- *Sales Force Composite*: aggregation of estimates made by sales agents (vendors)
- *Polls* among customers
- *Opinions* of managers
- *Delphi*: ask the question, gather individual opinions, share the opinions anonymously. Re-ask the question, repeat the procedure until consensus.

Objective models

Objective models are based on mathematical models. Three high-level phases:

- *Model specification* (*identification*) involves the choice of the forecast technique to use.
- *Model fitting* (*parameter setting*) given the model sets its parameters to maximize coherence of forecasts with actual data.
- *Model diagnosis* (*validation*) determines the level of coherence between actual and forecast data.

Econometric models

An econometric model yields a theory on how different economic factors interact among themselves.

The model tries to predict how a change of a variable affects the future values taken by other variables.

Typically defined along these phases:

1. Choice of the variables to include in the model;
2. Separation between endogenous and exogenous variables;
3. A priori definition of the causal relationships among the variables;
4. Specification of quantitative equations;
5. Parameter setting;
6. Forecast.

Econometric models, example

Relation between national income and consumption, as proposed by J.M. Keynes (*1936*)

$$c = \mu + \beta y$$

Where c is the consumer spending, y is the real disposable income and μ and β are parameters.

$\beta = {}^{dc}/_{dy}$ is called marginal propensity to consume, it is positive and less than 1, while μ, autonomous consumption, is strictly positive.

The model has (potentially) observable quantities, consumptions and income (the variables), and other non observable ones, the parameters μ and β.

Healthcare model, COVID, SIR

- Susceptibles decrease (get infected) each day by $\beta \cdot I \cdot S/n$:

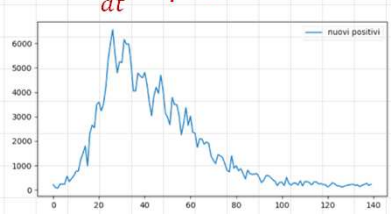
$$\frac{dS}{dt} = -\beta \cdot I \cdot S/n$$

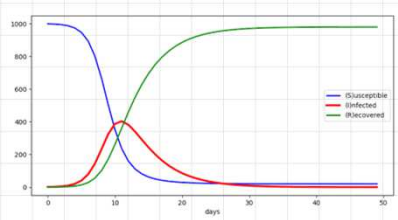
- Each day a percentage γ of Infected recover. Total; γI
  Total daily *variation of infected*:

$$\frac{dI}{dt} = \beta \cdot I \cdot \frac{S}{n} - \gamma \cdot I$$

- Total daily *variation of Recovered*:

$$\frac{dR}{dt} = \gamma \cdot I$$





Time series

A time series is simply a set of time-indexed values and represents the dynamics of a process over time.

In the model, we assume to have n observations coming from as many dependent random variables.

Objective of the study of time series is to find patterns in past data, which can be assumed to repeat in the future.

Date	*Open*
10/5/2021	7011.01
10/4/2021	7027.07
10/1/2021	7086.42
9/30/2021	7108.16
9/29/2021	7028.1
9/28/2021	7063.4
9/27/2021	7051.48
9/24/2021	7078.35
9/23/2021	7083.37
9/22/2021	6980.98
9/21/2021	6903.91
9/20/2021	6963.64

Time series, examples

Can be found in any economic sectors

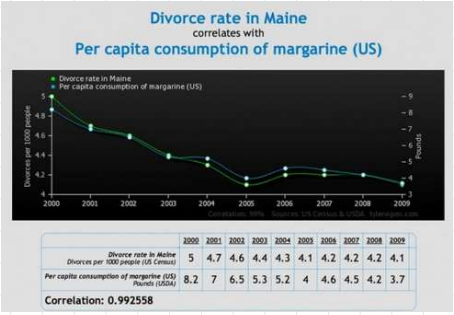
Organization	Series
Sales agent	Daily, weekly, monthly, … sales
Manufacturing company	Monthly labor costs (hours or euros)
Manufacturing company	Trimestral sales (units or euros)
Local government	Fiscal income (monthly)
Local government	Weekly building permits
Parish	Attendance at masses, weekly
University	Num. of students enrolled (absolute, percent)
Bank	Loans granted, weekly (number, euro)
agricultural company	Production per hectare, yearly
hospital	Man-days of hospitalization, monthly
Family	Telephone costs, bimestrial

Time series

A *time-series plot* (timeplot, time series graph) is a bidimensional chart of the time series.

The vertical axis is the variable of interest, the horizontal axis the time units.

Ex.

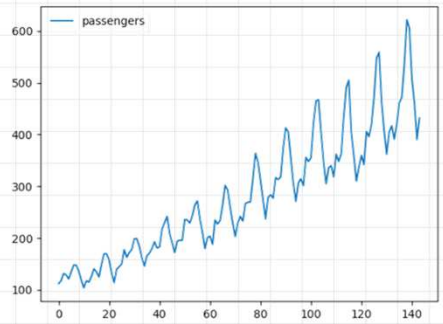


	2000	2001	2002	2003	2004	2005	2006	2007	2008	2009
Divorce rate in Maine Divorces per 1000 people (US Census)	5	4.7	4.6	4.4	4.3	4.1	4.2	4.2	4.2	4.1
Per capita consumption of margarine (US) Pounds (USDA)	8.2	7	6.5	5.3	5.2	4	4.6	4.5	4.2	3.7

http://www.tylervigen.com/spurious-correlations

THE time series

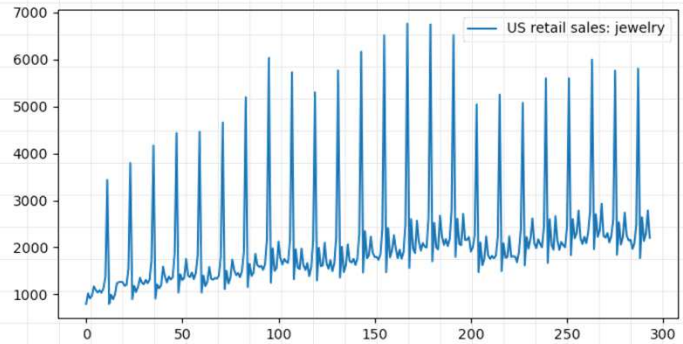
International airline passengers: monthly totals in thousands. Jan 49 – Dec 60 (G.E.P. Box, G.M. Jenkins, 1976).




```
df = pd.read_csv("BoxJenkins.csv")
plt.plot(df.Passengers,label="passengers")
plt.legend()
```

Another time series

Jewelry market sales, USA (millions of $).



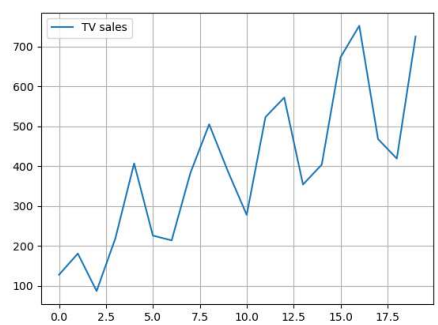

```
df = pd.read_csv("gioiellerie.csv")
fig = plt.figure(figsize=(8,4))
plt.plot(df.sales,label="US retail sales: jewelry")
plt.legend()
```


17

Slides example (fil rouge)

Daily tv sales

t	*year*	*trim.*	*sales*
1	2004	1	128
2		2	181
3		3	87
4		4	219
5	2005	1	407
6		2	226
7		3	214
8		4	383
9	2006	1	505
10		2	387
11		3	278
12		4	523
13	2007	1	572
14		2	354
15		3	404
16		4	673
17	2008	1	752
18		2	468
19		3	419
20		4	725




```
df = pd.read_csv("FilRouge.csv")
plt.plot(df.sales,label="TV sales")
plt.grid()
plt.legend()
plt.show()
```


18

Time series and forecast

Forecasting a time series wants to predict the values a time series is going to take in the future.

Two types of time series forecasting:

- *Univariate* Time Series Forecasting: uses only the previous values of the series to predict its future values.
- *Multi Variate* Time Series Forecasting: use predictors other than the series (a.k.a exogenous variables) to forecast.

Time series forecasts

Two main approaches:

1. *Statistics*. Based on a well-defined theoretical corpus, yields justified results.
2. *Machine learning*, neural or otherwise. More recent, often provides better results but without justification of the applied models.

Time series forecasts

In statistical methods, after training on historical data, an equation is used to present the relationship between load and its corresponding factors.

In artificial intelligence methods, the human way of thinking is copied. Artificial intelligence-based algorithms have been introduced based on expert system, evolutionary programming, fuzzy systems, etc. but by far the most effective ones are machine learning methods (neural or otherwise).

Statistical methods such as regression models, Box Jenkins models, exponential smoothing and Kalman filters have difficulties in identifying complex nonlinear relationships among factors but are explainable in their results.

Regression analysis

Regression analysis tries to estimate the relationships among variables, usually the relationship between a dependent variable and one or more independent variables (or '*predictors*'). It estimates a function.

Regression analysis is not limited to prediction and forecasting, but more generally wants to determine the value of the dependent variable when the independent variables are fixed.

It can take the form of a function relating the independent variables to the dependent one.

Regression, forecast, prediction

Regression is the general term often used in the ML literature to refer to prediction tasks.

- Forecasting: out of sample observations,
- Prediction: in sample observations (a model of the data).

Predicted values are calculated for observations in the sample used to estimate the regression.

Forecast is made for some date beyond the data used to estimate the regression, the data of the forecasted variable are not in the sample used to estimate the regression.

Residuals: difference between the actual value of Y and its predicted value for observations in the sample.

Forecast error: difference between future value of Y, not contained in the estimation sample, and the forecast of the future value.

Time series decomposition

Time series decomposition deconstructs a time series into several components, each representing one of the underlying categories of patterns.

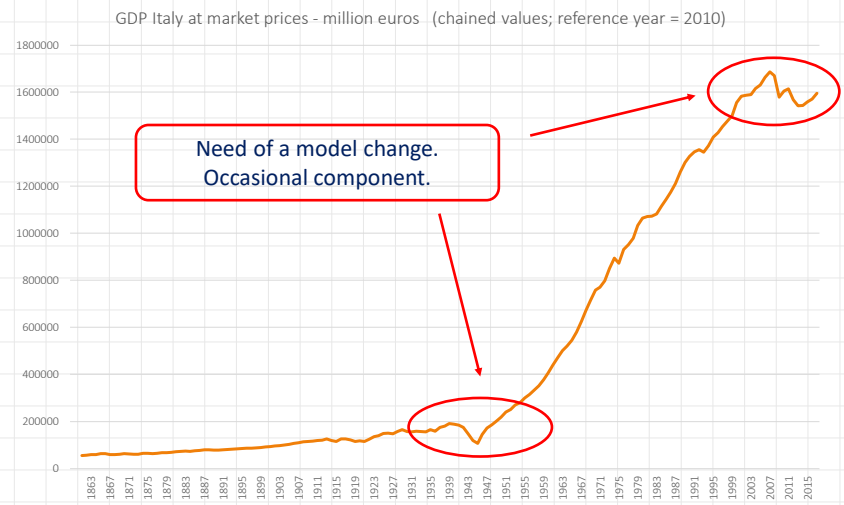
A common decomposition is into:

- Trend component
- Seasonal component
- Cyclical component
- Random component
- (occasional component)

A common model is STL (seasonal decomposition of time series by Loess, where Loess is short for Local Regression), that decomposes the series into seasonal, trend and irregular, where the cyclical component is included in the trend component plot.

Occasional discontinuities

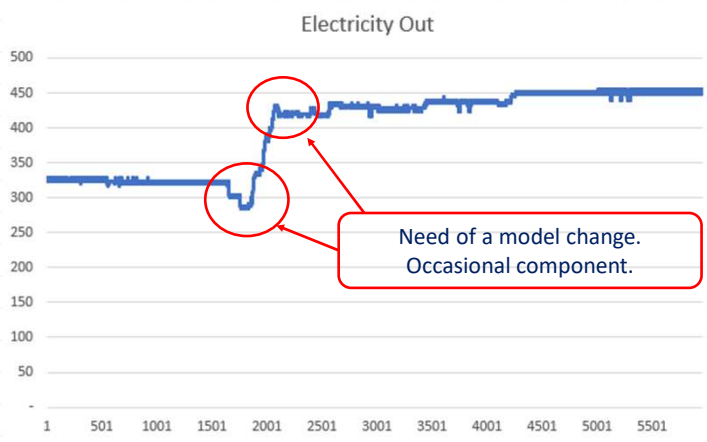
GNP Italy 1861 - 2017



25

Occasional discontinuities

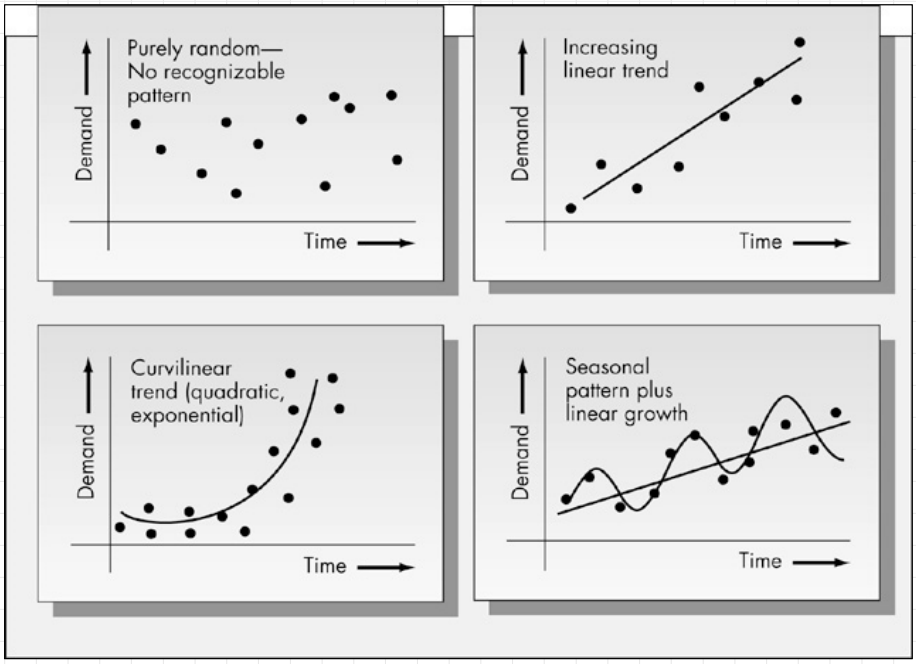
Power energy forecast



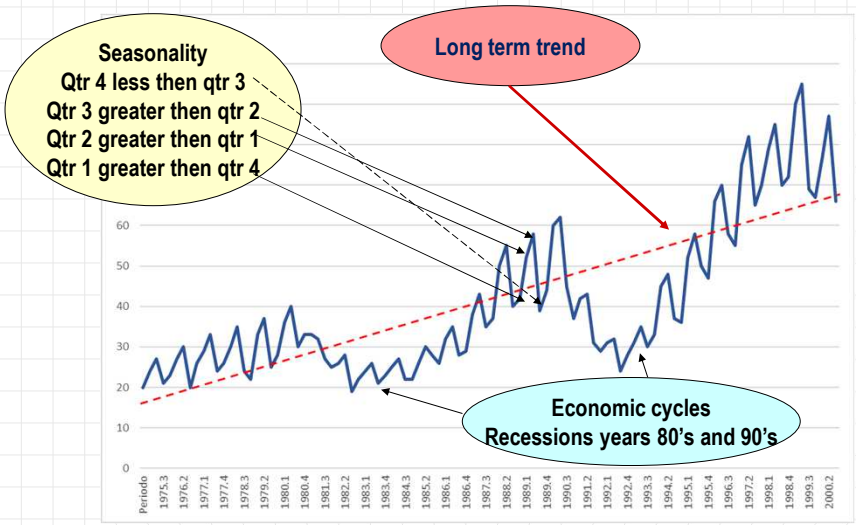
26

Time series decomposition

- *Trend component* is typically a persistent increasing or decreasing direction in the data. Usually modelled by a linear, quadratic or exponential function.
- *Seasonal component* includes repetitive patterns, typically over periods of weeks, months or years.
- *Cyclical component* reflects repeated but non-periodic fluctuations (non seasonal) lasting at least two years, typically due to changes in economic conditions (recessions, expansions).
- *Random component* (irregular, noise): random, irregular influences
- *Occasional component*: rare, due to war events, important technological innovations, political crises, etc..

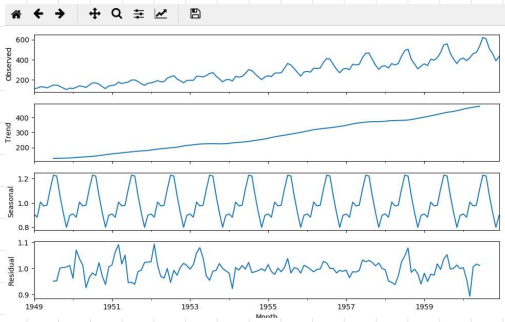


Camper sales, USA



Decomposition, python

```
from statsmodels.tsa.seasonal import seasonal_decompose
plt.rcParams['figure.figsize'] = (10.0, 6.0)
df = pd.read_csv('BoxJenkins.csv',usecols=["Passengers"])
ds = df[df.columns[0]]            # converts to series
result = seasonal_decompose(ds, model='multiplicative',period=12)
result.plot()
plt.show()
```



Multiple seasonalities

Several packages can identify multiple seasonalities:

- MSTL: https://www.statsmodels.org/dev/examples/notebooks/generated/mstl_decomposition.html
- Facebook's Prophet: https://pypi.org/project/prophet/
- BATS and TBATS: https://github.com/intive-DataScience/tbats

. . .

Time series model

Hypothesis: the observed y_t time series values result from the combination of different components.

- Long term trend μ_t
- Seasonal component s_t (period L)
- cyclical component c_t
- Random residual, irregular r_t.

The components are composed into different types of models:

Additive models: $y_t = \mu_t + s_t + c_t + r_t$

Multiplicative models: $y_t = \mu_t \times s_t \times c_t \times r_t$

Mixed models: $y_t = \mu_t \times s_t \times c_t + r_t$

An additive model is used when the variations around the trend do not vary with the level of the time series, a multiplicative model is used if the trend is proportional to the level of the time series.

Fil rouge, empirical STL

Fil rouge example, step by step empirical STL (Seasonal Trend with Local regression - Loess), with projection into the future.

Linear regression: computes a linear function that minimizes the global distance from dataset points.

Moving average: creates a series of averages of different subsets of the full data set. Subset are modified by "shifting forward“: excluding the first number of the series and including the next value in the subset.

Centered moving average (CMA): aligns each moving average with the middle of the time span of the observations that it averages. If computed on an even number of terms, need to average the averaged values.

Linear regression

We look for the interpolation line

$y = \theta_0 + \theta_1 x$ that minimizes the

mean square error on all data pairs (x_i, y_i):

$$e(\boldsymbol{\Theta}) = \sum \left((x_i, y_i) - (x_i, \theta_0 + \theta_1 x_i)\right)^2$$

Analytically[1], having the average values $\bar{x}$ and $\bar{y}$: coefficients

$$\theta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} \qquad \left(= \frac{n \cdot \sum x_i y_i - \sum x_i \cdot \sum y_i}{n \cdot \sum x_i^2 - (\sum x_i)^2}\right)$$

and:

$$\theta_0 = \bar{y} - \theta_1 \cdot \bar{x} \qquad \left(= \frac{\sum y_i \cdot \sum x_i^2 - \sum x_i \cdot \sum x_i y_i}{n \cdot \sum x_i^2 - (\sum x_i)^2}\right)$$

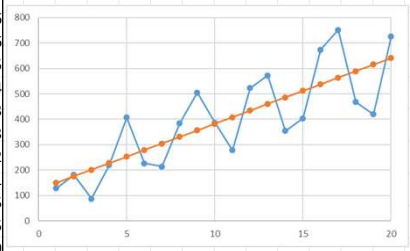
Python:

```
th1 = ((X*y).mean() - X.mean()*y.mean())/((X**2).mean()-(X.mean())**2)
th0 = y.mean() - th1*X.mean()
```

[1] You can derive this by yourself or read the derivation f.e. in https://math.stackexchange.com/questions/131590/derivation-of-the-formula-for-ordinary-least-squares-linear-regression

Linear trend model

x	*y*	*y-\bar(y)*	*x-\bar(x)*	*numerat*	*denom*	*regr*
1	128	-267.25	-9.5	2538.875	90.25	149.3142857
2	181	-214.25	-8.5	1821.125	72.25	175.2022556
3	87	-308.25	-7.5	2311.875	56.25	201.0902256
4	219	-176.25	-6.5	1145.625	42.25	226.9781955
5	407	11.75	-5.5	-64.625	30.25	252.8661654
6	226	-169.25	-4.5	761.625	20.25	278.7541353
7	214	-181.25	-3.5	634.375	12.25	304.6421053
8	383	-12.25	-2.5	30.625	6.25	330.5300752
9	505	109.75	-1.5	-164.625	2.25	356.4180451
10	387	-8.25	-0.5	4.125	0.25	382.306015
11	278	-117.25	0.5	-58.625	0.25	408.193985
12	523	127.75	1.5	191.625	2.25	434.0819549
13	572	176.75	2.5	441.875	6.25	459.9699248
14	354	-41.25	3.5	-144.375	12.25	485.8578947
15	404	8.75	4.5	39.375	20.25	511.7458647
16	673	277.75	5.5	1527.625	30.25	537.6338346
17	752	356.75	6.5	2318.875	42.25	563.5218045
18	468	72.75	7.5	545.625	56.25	589.4097744
19	419	23.75	8.5	201.875	72.25	615.2977444
20	725	329.75	9.5	3132.625	90.25	641.1857143



avg	395.2		25.88796
y	5	*a*	992
avg			123.4263
x	10.5	*b*	158

Seasonality component

Seasonality induces periodic fluctuations around the trend.

We look for a seasonality coefficient against which to multiply trend data.

First, we need to find out the periodicity (wavelength), the number of seasons.

Several methods, here autocorrelograms.

The correlation of two time series can be quantified by Pearson's index (parametric) or Spearman's (nonparametric).

Correlograms

A correlogram, or autocorrelation plot, is a chart that represents the autocorrelation of a time series as a function of the lag used for computing the autocorrelation.

To define the correlogram, we consider the correlations between the series and itself shifted by each of K periods;

F.e., given Y = (y_1, ... , y_n) we construct the table below and we examine the *K* correlations between column y_t and each of the K columns y_{t-k}

Y_t	Y_{t-1}	Y_{t-2}	Y_{t-3}	...	Y_{t-K}
Y_1					
Y_2	Y_1				
Y_3	Y_2	Y_1			
Y_4	Y_3	Y_2	Y_1		
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
Y_{T-2}	Y_{T-3}	Y_{T-4}	Y_{T-5}	$\vdots$	Y_{T-K-2}
Y_{T-1}	Y_{T-2}	Y_{T-3}	Y_{T-4}	$\vdots$	Y_{T-K-1}
Y_T	Y_{T-1}	Y_{T-2}	Y_{T-3}	$\vdots$	Y_{T-K}

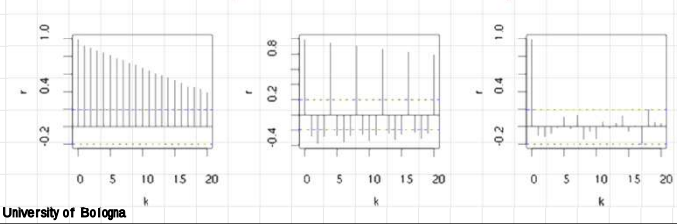
Correlograms

We start from the (K+1)-th row, to compare equal length series.

We compute a value for r_k letting k range from 1 to K and computing the correlation (here Pearson, but could be Spearman) r_k between column Y_t , average $\overline{Y}_t$, and the column of the lagged variable Y_{t-k}:

$$r_k = \frac{\sum_{t=k+1}^{T}(Y_t - \overline{Y}_t)(Y_{t-k} - \overline{Y}_t)}{\sum_{t=k+1}^{T}(Y_t - \overline{Y}_t)^2}$$

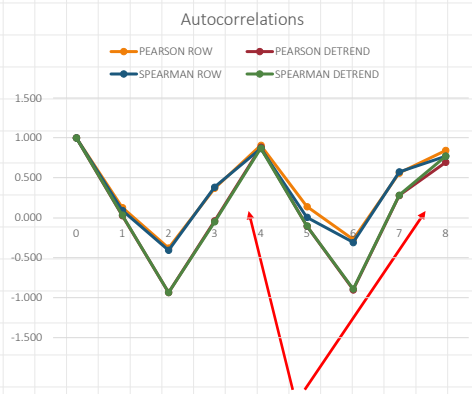
Pairs of values (k, r_k) are shown on a cartesian chart, with lags on the x-axis and the corresp. correlation on the y-axis:



Fil rouge, correlogram

128								
181	128							
87	181	128						
219	87	181	128					
407	219	87	181	128				
226	407	219	87	181	128			
214	226	407	219	87	181	128		
383	214	226	407	219	87	181	128	
505	383	214	226	407	219	87	181	128
387	505	383	214	226	407	219	87	181
278	387	505	383	214	226	407	219	87
523	278	387	505	383	214	226	407	219
572	523	278	387	505	383	214	226	407
354	572	523	278	387	505	383	214	226
404	354	572	523	278	387	505	383	214
673	404	354	572	523	278	387	505	383
752	673	404	354	572	523	278	387	505
468	752	673	404	354	572	523	278	387
419	468	752	673	404	354	572	523	278
725	419	468	752	673	404	354	572	523
	725	419	468	752	673	404	354	572
		725	419	468	752	673	404	354
			725	419	468	752	673	404
				725	419	468	752	673
					725	419	468	752
						725	419	468
							725	419
								725

PEARSON								
1.000	0.126	-0.380	0.370	0.907	0.139	-0.271	0.560	0.842
lag								
0	1	2	3	4	5	6	7	8



Seasonality of 4

Correlogram

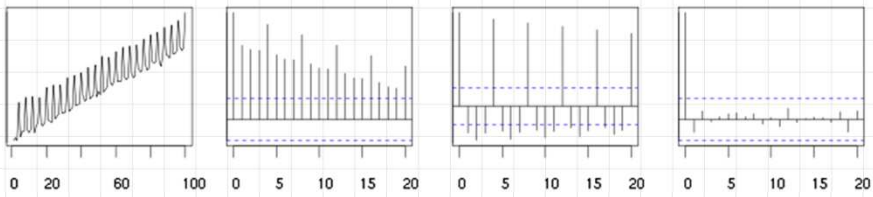
Correlograms can be widely different, but we have three typical cases:

Original data series

Original data series correlogram

Residuals after detrending

Residuals after detranding and deseasoning



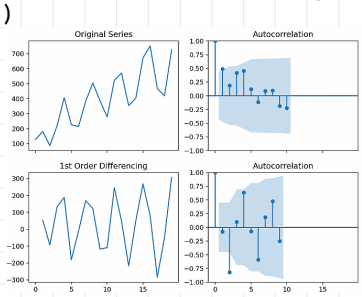
Python, ACF

```python
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# Import data
df = pd.read_csv('FilRouge.csv', usecols=[1], names=['value'], header=0)

# Original Series
plt.rcParams.update({'figure.figsize':(9,7), 'figure.dpi':120})
fig, axes = plt.subplots(2, 2, sharex=True)
axes[0, 0].plot(df.value); axes[0, 0].set_title('Original Series')
plot_acf(df.value, ax=axes[0, 1])

# 1st Differencing
axes[1, 0].plot(df.value.diff()); axes[1, 0].set_title('1st Order Differencing')
plot_acf(df.value.diff().dropna(), ax=axes[1, 1])
plt.show()
```



Seasonality component

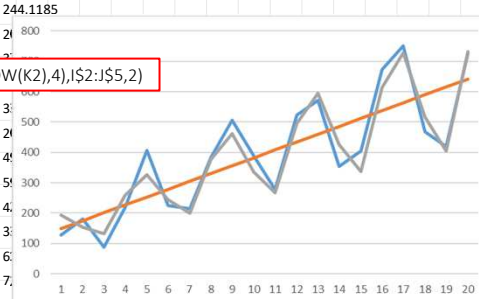
Seasonality induces periodic fluctuations around the trend.
We look for a coefficient against which to multiply trend data.

x	y	regr	Relative elong		avg seas elong	*coeff*	seasonal
1	128	149.3143					
2	181	175.2023					
3	87	201.0902	0.432641615	0	0.657315894	0.657316	132.1798
4	219	226.9782	0.964850388	1	1.142187151	1.142187	259.2516
5	407	252.8662	1.609547087	2	1.292339829	1.29234	326.789
6	226	278.7541	0.810750304	3	0.875748574	0.875749	244.1185
7	214	304.6421	0.702463633			0.657316	2
8	383	330.5301	1.158744782				
9	505	356.418	1.416875512				
10	387	382.306	1.012278083			0.875749	3
11	278	408.194	0.681048742			0.657316	2
12	523	434.082	1.204841607			1.142187	4
13	572	459.9699	1.243559566			1.29234	5
14	354	485.8579	0.728608105			0.875749	4
15	404	511.7459	0.789454352			0.657316	3
16	673	537.6338	1.251781336			1.142187	6
17	752	563.5218	1.334464778			1.29234	7
18	468	589.4098	0.794014657			0.875749	5
19	419	615.2977	0.68097113			0.657316	404.445

=SUMPRODUCT((MOD(ROW($H$2:$H$21),4)=I2)*($H$2:$H$21))/(COUNT($H$2:$H$21)/4)

Will be MOD(row(.),4) = 0

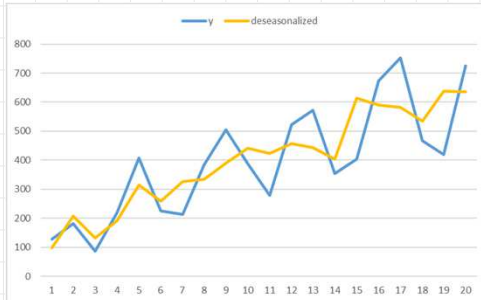
=VLOOKUP(MOD(ROW(K2),4),I$2:J$5,2)



Fil rouge, deseasoning

In order to eliminate the effect of s_t, we must divide by s_t. We are left with trend and random components.

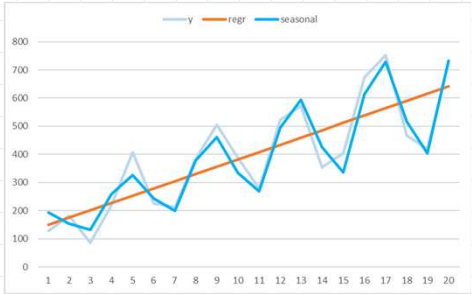
t	y_t	s_t	*Deseason..*
1	128	1.29	99.05
2	181	0.88	206.68
3	87	0.66	132.36
4	219	1.14	191.74
5	407	1.29	314.93
6	226	0.88	258.06
7	214	0.66	325.57
8	383	1.14	335.32
9	505	1.29	390.76
10	387	0.88	441.91
11	278	0.66	422.93
12	523	1.14	457.89
13	572	1.29	442.61
14	354	0.88	404.23
15	404	0.66	614.62
16	673	1.14	589.22
17	752	1.29	581.89
18	468	0.88	534.40
19	419	0.66	637.44
20	725	1.14	634.75



Fil rouge, noise removal

We can remove noise by computing $y_t = \mu_t \times s_t$ (here $T_t \times s_t$)

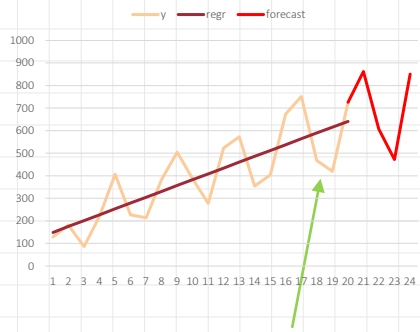
t	y_t	*st*	*Tt*	*st Tt*
1	128	1.38	149.314	192.96
2	181	0.82	175.202	153.43
3	87	0.64	201.090	132.18
4	219	1.11	226.978	259.25
5	407	1.38	252.866	326.79
6	226	0.82	278.754	244.12
7	214	0.64	304.642	200.25
8	383	1.11	330.530	377.53
9	505	1.38	356.418	460.61
10	387	0.82	382.306	334.80
11	278	0.64	408.194	268.31
12	523	1.11	434.082	495.80
13	572	1.38	459.970	594.44
14	354	0.82	485.858	425.49
15	404	0.64	511.746	336.38
16	673	1.11	537.634	614.08
17	752	1.38	563.522	728.26
18	468	0.82	589.410	516.17
19	419	0.64	615.298	404.44
20	725	1.11	641.186	732.35



Fil rouge, forecast *(empirical)*

Having the coefficients of the trend and seasonal components, we can project the model onto future periods.

x	y	regr	s_t	forecast
1	128	149.314	1.29	
2	181	175.202	0.88	
3	87	201.090	0.66	
4	219	226.978	1.14	
5	407	252.866	1.29	
6	226	278.754	0.88	
7	214	304.642	0.66	
8	383	330.530	1.14	
9	505	356.418	1.29	
10	387	382.306	0.88	
11	278	408.194	0.66	
12	523	434.082	1.14	
13	572	459.970	1.29	
14	354	485.858	0.88	
15	404	511.746	0.66	
16	673	537.634	1.14	
17	752	563.522	1.29	
18	468	589.410	0.88	
19	419	615.298	0.66	
20	725	641.186	1.14	725
21		667.074	1.29	862.09
22		692.962	0.88	606.86
23		718.850	0.66	472.51
24		744.738	1.14	850.63



Variance grows too much. A log transform preprocessing was in order.

Sampling

In any learning process, performance should be estimated with out-of-sample validation, withholding some of the sample data from model identification (training phase), then using the model to make forecasts for the hold-out data (test data)

Need to split the data into training and testing buckets. the testing dataset should never be used in the learning step

k-fold cross validation repeats this process by repeatedly splitting the data into k groups, each given a chance to be a held out model. It cannot be used when learning on data series.

Prediction on train values are called *fitted values* and the corresponding errors are called *residuals*.

Sampling

Errors on train data can be much smaller than those on test data, in case of *overfitting*.

Overfitting is likely to occur when

1. a model with a large number of parameters has been fitted to a too small sample of data
2. the model has been selected from a large set of potential models precisely by minimizing the RMSE in the estimation period.

Train-Test Split

You must split your dataset into training and testing subsets.

Your model can be prepared on the training dataset and predictions can be made and evaluated for the test dataset.

This can be done by selecting an arbitrary split point in the ordered list of observations and creating two new datasets. Depending on the amount of data you have available and the amount of data required, you can use splits of 50-50, 70-30 and 90-10.

FilRouge: we should have reserved the last data for testing!

```
ds = pd.read_csv('BoxJenkins.csv', header=0)
X = ds.Passengers.values
train_size = int(len(X) * 0.66)
train, test = X[0:train_size], X[train_size:len(X)]
print('Observations: %d' % (len(X)))
print('Training Observations: %d' % (len(train)))
print('Testing Observations: %d' % (len(test)))
pyplot.plot(train)
pyplot.plot([None for i in train] + [x for x in test])
pyplot.show()
```